

Final Project: DonkeyCar Agentic Development

Talal Jeddawi (*CSE*), Leo Okdemir (*HDSI*)

Must Haves

- Working **line-following behavior**, driven entirely by agent-written CV/control code
- **Lane following**: upgrade from single-line tracking to full lane navigation
- **Two-way road navigation**: detect an oncoming robot and avoid collision
- **Adaptive speed control**: slow down on curves, speed up on straights, based on line curvature rather than constant throttle
- **Documentation** of the iteration process

Nice to Haves

- **Chaining all three stages into one integrated run**, line following → lane following → two-way navigation executed as a single mission
- **Static obstacle avoidance**, detect and steer around a stationary object in-lane

What We Did

- Used Claude as the coding agent for the full CV/control pipeline: we wrote mission prompts, Claude wrote the code, we tested on the car, gave feedback, Claude revised
- Used Claude's web browser (claude.ai) to draft and refine the actual prompts we fed to the Claude Code agent in VS Code, treating prompt-writing itself as an iterative step, not just the code
- **Oscillation-based dashed-line detector:** since the dashed yellow line periodically disappears between segments, raw frame-by-frame detection caused the tracked x-position to swing/drop out; instead of reacting to each raw reading, the detector searches left and right from the last known center position, smoothing/holding through the gaps until the next dash segment is picked back up
- Achieved full-autonomous line tracking with self-adjusting speed by end of project

What Worked

- **Single-line following:** agent-written HSV threshold + histogram/PID control tracked the yellow line reliably
- **Lane tracking** using the yellow dashed centerline plus white edge lines for cross-checking, once tuned
- **Adaptive speed control:** car slowed on curves and sped up on straights based on detected line offset/curvature
- **End result:** full autonomous lap, staying in-track, with self-adjusting speed

Video

(M)ode Auto (S)teer

1 2 3 4 5



Note: physical game controller overrides web control.



Joystick Gamepad Device Tilt

Throttle: Maximum 100%

What Didn't Work As Expected

1. **VESC hardware issues**, had to restart the whole car setup process several times. Initially misdiagnosed as a software/driver issue (serial port errors looked like a config or code problem), before identifying it as a hardware/connection fault with the VESC itself.
2. **White edge-line noise** using the white lane edges for cross-checking introduced a lot of false-positive/noisy detections, made PID tuning much harder than single yellow-line tracking.
 - *Why*: white boundary detection was more sensitive to lighting variation and reflections than the yellow line mask; combining two color/edge signals amplified small errors.
 - *How we'd solve it*: Tighter/adaptive tunable parameters per-lighting-condition
3. **Repository File Corruption**
4. **Agent Missing Hardware Context**

If We Had Another Week

- Push further into two-way object navigation
- Revisit white-edge detection with lighting-invariant preprocessing rather than raw HSV thresholding
- Further refine the must-haves
- Work on more prompt engineering

Gantt Chart Changes / Lessons Learned

- Original timeline assumed line following → lane following → two-way navigation would progress roughly linearly; VESC hardware failures and agent mishaps ate significant time early/mid-project that pushed later stages back
- Lesson: Utilize images collected by running the the car manually to guide agentic development
- Lesson: prompt quality mattered as much as agent code quality — drafting/refining prompts in claude.ai before handing them to the VS Code agent reduced wasted revision loops

